

Dynamic Workflows com AI Agents e Sub-Agents: Padrões de Orquestração e Frameworks para Produção

OpenClaw Agency

13 de julho de 2026

[AUTOR]^{1*}

¹ [AFILIAÇÃO DO AUTOR — DEPARTAMENTO, INSTITUIÇÃO, CIDADE, PAÍS]

Autor correspondente: [EMAIL]

Resumo

A adoção de sistemas multi-agente evoluiu de experimentos de laboratório para implantações empresariais em escala entre 2024 e 2026, impulsionada pela maturidade dos modelos de linguagem de grande porte (LLMs). O presente artigo apresenta uma síntese crítica e uma análise arquitetônica dos padrões fundamentais de orquestração de agentes de inteligência artificial (AI Agents) e sub-agentes (Sub-Agents) em ambientes de produção. Discute-se, detalhadamente, seis padrões de orquestração — Orchestrator-Worker (Supervisor), Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff (Routing), Group Chat e Magentic (Planning-Ledger) — bem como os três padrões de interação de sub-agentes (síncrono, assíncrono e agendado) e o papel central da compressão de contexto. Adicionalmente, analisa-se a maturidade, os diferenciais e a adequação dos principais frameworks do ecossistema — LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework e Spring AI Agent Utils — por meio de uma matriz de compatibilidade estruturada e de uma matriz de critérios de seleção. O texto examina, ainda, aplicações setoriais documentadas, a modelagem quantitativa de custos e latência, os riscos e limitações da abordagem e um roteiro de adoção em produção. O artigo conclui com diretrizes práticas de engenharia para adoção em produção, abrangendo gerenciamento de contexto, otimização de custos, segurança orientada ao princípio do menor privilégio, human-in-the-loop e observabilidade.

Palavras-chave: Agentes de IA. Workflows Dinâmicos. Orquestração Multi-agente. Padrões de Arquitetura. LangGraph. CrewAI.

Abstract

The adoption of multi-agent systems has evolved from laboratory experiments to enterprise-scale production deployments between 2024 and 2026, driven by the maturity of large language models (LLMs). This article presents a critical synthesis and an architectural analysis of the fundamental orchestration patterns for artificial intelligence agents (AI Agents) and sub-agents (Sub-Agents) in production environments. Six orchestration patterns are discussed in detail — Orchestrator-Worker (Supervisor), Sequential Pipeline, Parallel Fan-Out/Fan-In, Dynamic Handoff (Routing), Group Chat, and Magentic (Planning-Ledger) — as well as the three sub-agent interaction patterns (synchronous, asynchronous, and scheduled) and the central role of context compression. Additionally, the maturity, differentials, and suitability of the main ecosystem frameworks — LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework, and Spring AI Agent Utils — are analyzed through a structured compatibility matrix and a selection criteria matrix. The text examines, still, documented sectorial applications, the quantitative modeling of costs and latency, the risks and limitations of the approach, and a production adoption roadmap. The article concludes with practical engineering guidelines for production adoption, covering context management, cost optimization, security oriented to the principle of least privilege, human-in-the-loop, and observability.

— along with the three sub-agent interaction patterns (synchronous, asynchronous, and scheduled) and the central role of context compression. Furthermore, the maturity, differentials, and suitability of the main ecosystem frameworks — LangGraph, CrewAI, OpenAI Agents SDK, Claude Agent SDK, Microsoft Agent Framework, and Spring AI Agent Utils — are analyzed through a structured compatibility matrix and a selection-criteria matrix. The text also examines documented sector applications, the quantitative modeling of cost and latency, the risks and limitations of the approach, and a production adoption roadmap. The article concludes with practical engineering guidelines for production adoption, covering context management, cost optimization, security oriented to the principle of least privilege, human-in-the-loop, and observability.

Keywords: AI Agents. Dynamic Workflows. Multi-agent Orchestration. Architectural Patterns. LangGraph. CrewAI.

1 Introdução

A inteligência artificial generativa atingiu maturidade em que LLMs resolvem tarefas isoladas com alta competência. Problemas complexos do mundo real, porém, raramente são estanques: um processo empresarial típico envolve pesquisa, análise, redação, revisão, aprovação e publicação, cada etapa com habilidades e critérios distintos (VELLUM AI, 2025). Um modelo generalista tende a resultados medianos ao manter todo o contexto enquanto se especializa em cada subtarefa.

Daí emergem os sistemas multi-agente: múltiplos agentes especializados colaboram, cada um com seu modelo, ferramentas e instruções (MICROSOFT, 2026), tornando o sistema modular, auditável e resiliente, e reduzindo regressões em cascata. O valor central dos sub-agentes não é só paralelismo, mas compressão de contexto: bem projetados, mantêm o contexto do pai enxuto, reduzindo tokens em mais de 90% e prevenindo degradação por limite de contexto (EPSILLA, 2026).

O objetivo deste artigo é oferecer uma referência arquitetônica consolidada, mapeando padrões, mecanismos de sub-agentes, frameworks e diretrizes de engenharia para produção.

2 Fundamentos de Dynamic Workflows

Dynamic Workflows são arquiteturas nas quais múltiplos agentes colaboram de forma coordenada (ZYLOS RESEARCH, 2026). Diferente de pipelines estáticos, a topologia — quais agentes, em que ordem e frequência — é definida em tempo de execução, conforme o contexto e resultados parciais. A distinção estático/dinâmico é sobre *onde* a decisão de controle reside: em design ou delegada ao modelo.

O conceito apoia-se em três pilares (MICROSOFT, 2026; AWS, 2025): **Decomposição** (tarefa em subtarefas especializáveis); **Delegação** (à unidade mais qualificada por especialização, custo e segurança); **Coordenação** (agregação e integração dos resultados). A orquestração pode ser estática (topologia em design) ou dinâmica (emerge por decisões de LLMs); a maioria adota híbrido: estrutura predefinida para consistência, com roteamento e replanejamento dinâmicos (AWS, 2025).

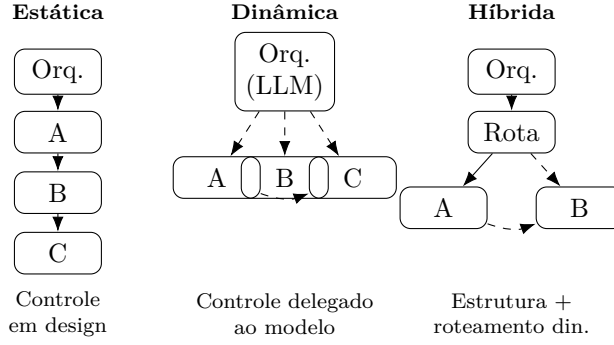


Figura 1: Esquema comparativo das topologias estática, dinâmica e híbrida de orquestração multi-agente, ilustrando o locus da decisão de controle.

3 Padrões de Orquestração em Produção

O ecossistema contemporâneo de engenharia de software voltado à inteligência artificial converge para seis padrões fundamentais de orquestração de agentes (MICROSOFT, 2026; ZYLOS RESEARCH, 2026). A seleção do padrão adequado deve considerar o grau de acoplamento entre as etapas, a necessidade de consenso entre especialistas e a tolerância a latência. A Tabela 1 sintetiza, ao final da seção, os trade-offs de cada padrão.

3.1 Orchestrator-Worker (Supervisor)

O padrão Orchestrator-Worker, frequentemente denominado Supervisor, é a arquitetura mais difundida em sistemas multi-agente comerciais. Um agente orquestrador central recebe a requisição do usuário, decompõe o problema em subtarefas, delega cada subtarefa a um agente especialista (*worker*), monitora o progresso de execução e sintetiza o resultado final (MICROSOFT, 2026). O orquestrador mantém o estado global e a responsabilidade pela consistência do produto entregue.

Este padrão é indicado para workflows complexos onde a decomposição requer julgamento contextual e os planos variam conforme a entrada. Por outro lado, adiciona 15% a 30% de latência e consome 20% a 40% mais tokens devido às idas e voltas com os *workers* (MICROSOFT, 2026). A centralização também introduz gargalo: a capacidade do orquestrador limita o *throughput*.

O Claude Agent SDK da Anthropic implementa este padrão como primitiva fundamental por meio da capacidade de expor agentes como ferramentas programáticas utilizando ‘agent.asTool()’ (ANTHROPIC, 2026a). A Microsoft, por sua vez, adota a mesma estrutura mapeando os agentes em grafos de fluxo de trabalho com transições explícitas (MICROSOFT, 2026). A escolha entre implementação via ferramentas programáticas (Anthropic) e via grafos explícitos (Microsoft) reflete um trade-off entre flexibilidade de composição e previsibilidade de execução.

3.2 Sequential Pipeline (Prompt Chaining)

Agentes encadeados em ordem linear estrita; a saída de um serve de entrada ao próximo (AWS, 2025; DEVARAJAN, 2026). Recomenda-se para dependências lineares claras (geração → revisão → verificação → formatação); evitar quando há *backtracking* (LUSHBINARY, 2026).

3.3 Parallel Fan-Out / Fan-In

Execução simultânea de múltiplos agentes por especialidade; consolidação em nó de agregação por votação ou sumarização (DIGITAL APPLIED, 2026). Ideal para análise multi-perspectiva sob restrição de tempo (PUCEK, 2026).

3.4 Dynamic Handoff (Routing)

Agentes decidem autonomamente quando transferir o controle a outro especialista; a transferência é definitiva, operando um agente por vez (OPENAI, 2026; MICROSOFT, 2026). Aplicável em triagem de atendimento; OpenAI expõe ‘handoff()’ (OPENAI, 2026), Claude via sub-agentes isolados (ANTHROPIC, 2026a).

3.5 Group Chat (Roundtable)

Múltiplos agentes em *thread* compartilhado; um *chat manager* define quem fala a cada turno, permitindo consenso (MICROSOFT, 2026). Rico em cruzamento de perspectivas, custa latência e difícil término (SUBAGENT.DEV, 2026).

3.6 Magentic (Planning-Ledger)

Para escopo aberto: o agente mantém um *ledger* dinâmico de tarefas, reavaliando e acionando sub-agentes conforme o contexto evolui (MICROSOFT, 2026). Usado em SRE e resposta a incidentes (ZYLOS RESEARCH, 2026).

Tabela 1. Síntese comparativa dos seis padrões de orquestração quanto a latência, consumo de tokens, adequação a consenso e previsibilidade.

Padrão	Latência	Tokens	Consenso	Previsibilidade	Indicado para
Orchestrator-Worker	Alta	Alta	Média	Alta	Decomposição com julgamento
Sequential Pipeline	Baixa	Baixa	Baixa	Muito alta	Dependências lineares
Parallel Fan-Out/Fan-In	Baixa	Alta	Alta	Média	Análise multi-perspectiva
Dynamic Handoff	Média	Média	Baixa	Média	Triagem e especialização
Group Chat	Alta	Alta	Alta	Baixa	Debate e design colaborativo
Magentic	Alta	Alta	Média	Baixa	Escopo aberto, incidentes

Fonte: elaborado pelo autor com base em Microsoft (2026), OpenAI (2026), Zylos Research (2026), Epsilla (2026) e Digital Applied (2026).

4 Sub-Agents: Padrões de Interação e Compressão de Contexto

Além dos padrões de orquestração de alto nível, a operação eficiente de sub-agentes em produção depende de três categorias operacionais de interação (EPSILLA, 2026):

Synchronous "Wait and See": o agente pai suspende sua execução e aguarda o retorno do sub-agente, comportando-se de forma análoga a uma chamada de função complexa. É o modo mais simples de raciocinar e depurar, porém limita o paralelismo do sistema. **Asynchronous "Fire and Forget"**: o agente pai delega uma tarefa e continua seu fluxo de execução em paralelo, útil para processamento assíncrono ou disparos de eventos. Exige mecanismos de correlação para consumir o resultado posteriormente. **Scheduled "Future Execution"**: o sub-agente é programado para executar em um momento futuro predeterminado ou sob gatilhos temporais específicos. É a base da automação orientada a eventos e da execução recorrente.

O insight central desse desacoplamento não reside apenas no processamento paralelo, mas na compressão do contexto de inferência. Ao isolar sub-tarefas em sub-agentes dotados de escopos de variáveis e ferramentas restritas, reduz-se o volume total de tokens acumulados nas janelas de contexto dos modelos principais, prevenindo a degradação da acurácia causada pelo fenômeno de "esquecimento no meio do contexto" (*lost in the middle*) (EPSILLA, 2026; VELLUM AI, 2025). O agente pai recebe apenas o resultado sintetizado da investigação, e não o rastro completo das ferramentas acionadas — o que explica a redução documentada de mais de 90% no consumo de tokens (EPSILLA, 2026).

A compressão de contexto opera em duas dimensões: (i) *temporal*, ao descartar ou resumir histórico obsoleto; e (ii) *espacial*, ao delimitar o escopo de ferramentas e variáveis de cada sub-agente. A combinação das duas torna viável a execução de tarefas de longa duração sem estouro da janela de contexto.

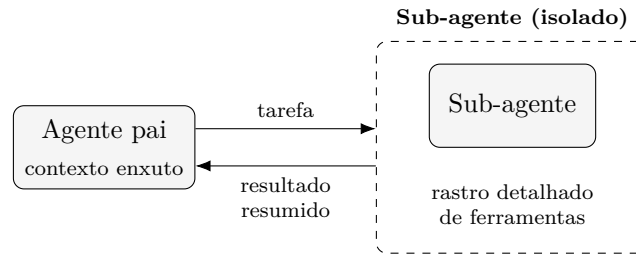


Figura 2: Representação do fluxo de compressão de contexto: o rastro detalhado das ferramentas permanece no sub-agente, enquanto o agente pai recebe apenas o resultado resumido.

5 Ecossistema de Frameworks e Ferramentas

O mercado de ferramentas para orquestração de agentes registrou rápida consolidação em torno de frameworks especializados (GHEWARE, 2026; JETTHOUGHTS, 2026). A escolha do framework não é meramente técnica: ela determina o modelo mental dos desenvolvedores, os limites de segurança disponíveis e a facilidade de observabilidade. A seguir, analisam-se os seis principais frameworks do ecossistema.

5.1 LangGraph

Extensão do LangChain para fluxos cíclicos modelados como grafos de estado, com controle preciso de execução (AGENT MARKET CAP, 2026). Diferenciais: *checkpointing*, *time-travel debugging*, integração LangSmith; limitação: curva íngreme (OPENAGENTS, 2026).

5.2 CrewAI

Metáfora organizacional de *crews* com papéis e memórias predefinidos (META INTELLIGENCE, 2025). Diferenciais: prototipagem rápida; limitação: controle granular limitado para dinâmicos (GHEWARE, 2026).

5.3 OpenAI Agents SDK

Foco em ‘handoffs’ simplificados, guardrails declarativos e *tracing* (OPENAI, 2026).

5.4 Claude Agent SDK

Foco em MCP e execução integrada a IDEs, com o *harness* do Claude Code, sub-agentes isolados e paralelização nativa (ANTHROPIC, 2026b; WINBUZZER, 2026).

5.5 Microsoft Agent Framework

Sucessor corporativo do AutoGen, integrado ao Azure, com *workflow graphs* e ênfase em conformidade (ROCKB, 2026).

5.6 Spring AI Agent Utils

Voltado ao ecossistema Java/Spring Boot, com *Task tools* para isolamento, *multi-model routing* e A2A (TZOLOV, 2026).

5.7 Matriz de Compatibilidade de Frameworks

A Tabela 2 apresenta a avaliação de compatibilidade nativa entre os frameworks analisados e os padrões de orquestração discutidos, consolidada a partir da literatura técnica revisada.

Tabela 2. Matriz de compatibilidade entre frameworks e padrões de orquestração

Fra- mework	Orches- trator	Sequen- tial	Parallel	Handoff	Group Chat	A2A	MCP
Lang- Graph CrewAI	Nativo	Nativo	Nativo	Via nós	Via <i>custom</i>	—	Via LangChain
OpenAI Agents SDK	Hierár- quico	Nativo	Nativo	Limitado	Não suportado	Sim	Comunitá- rio
Claude Agent SDK	Agent-as- Tool	Manual	Via <i>handoffs</i>	Nativo	Não suportado	—	Sim
MS Agent Fra- mework	Agent tool	Programá- tico	Sub- agentes	Sub- agentes	Agent Teams	Planejado	Nativo
Spring AI Agent Utils	<i>Workflow graphs</i>	<i>Workflow graphs</i>	Nativo	Nativo	<i>GroupChat</i> legado	Sim	<i>Built-in</i>
	<i>Task tool</i>	Programá- tico	<i>Task tool</i>	<i>Task tool</i>	Não suportado	Sim	Sim

Fonte: adaptado de Microsoft (2026), OpenAI (2026), Anthropic (2026a, 2026b), Tzolov (2026), RockB (2026) e Zylos Research (2026).

5.8 Critérios de Seleção de Framework

A Tabela 3 propõe um critério estruturado de seleção, útil para arquitetos que enfrentam a decisão de adoção. Cada critério deve ser ponderado pelos requisitos do projeto.

Tabela 3. Matriz de critérios para seleção de framework

Critério	Peso sugerido	Lang-Graph	CrewAI	OpenAI SDK	Claude SDK	MS AF	Spring AI
Controle de estado	Alto	●●●	●●○	●●○	●●○	●●●	●●○
Ergonomia/Prototipação	Médio	●●○	●●●	●●○	●●○	●●○	●●○
Segurança/Conformidade	Alto	●●○	●●○	●●○	●●○	●●●	●●○
Observabilidade	Alto	●●●	●●○	●●●	●●○	●●○	●●○
Ecossistema/Integração	Médio	●●●	●●○	●●○	●●○	●●●	●●●

Legenda: ●●● forte; ●●○ moderado; ●○● limitado. Fonte: elaborado pelo autor com base em Agent Market Cap (2026), OpenAgents (2026), Gheware (2026), RockB (2026) e Shakudo (2026).

6 Aplicações Setoriais e Estudos de Caso

A adoção de *dynamic workflows* não é meramente teórica: casos de produção documentados ilustram a aplicabilidade dos padrões apresentados em domínios diversos. A análise desses casos permite extrair princípios de design transferíveis.

No setor jurídico, um escritório utiliza um pipeline sequencial de quatro agentes para geração de contratos — seleção de template, customização de cláusulas, verificação de *compliance* e avaliação de risco (SHAKUDO, 2025) — refletindo a natureza regulatória e a necessidade de determinismo.

No financeiro, a análise de investimentos emprega Parallel Fan-Out/Fan-In, com agentes em paralelo para análises fundamentalista, técnica, de sentimento e ESG, convergindo para um relatório consolidado (PUCEK, 2026). No atendimento ao cliente, o Dynamic Handoff usa um agente de triagem que transfere para especialistas em suporte, faturamento ou retenção (OPENAI, 2026), otimizando contexto por especialista.

Na SRE, o padrão Magentic sustenta a resposta a incidentes, onde o agente analisa logs, delega diagnósticos e aplica correções à medida que o sistema se revela (ZYLOS RESEARCH, 2026). Em produção de conteúdo técnico, o Orchestrator-Worker coordena rascunho, revisão, verificação de fatos e formatação (VELLUM AI, 2025), demonstrando como a decomposição com julgamento contextual supera a execução monolítica.

7 Modelagem de Custos e Latência

A viabilidade econômica depende de modelagem cuidadosa, pois a orquestração introduz overhead sobre a execução mono-agente. O padrão Orchestrator-Worker adiciona 15%–30%

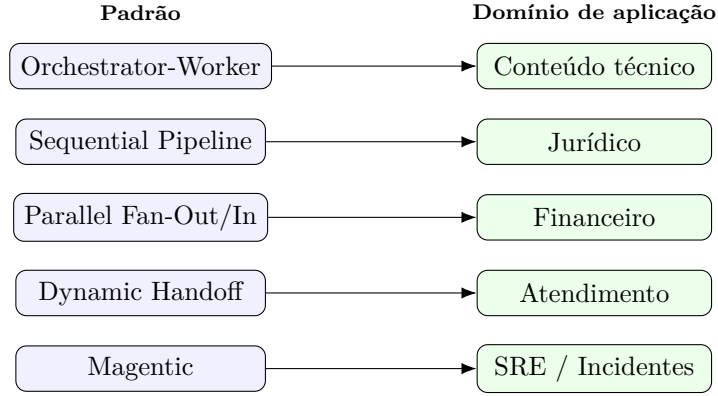


Figura 3: Mapa de alinhamento entre padrões de orquestração e domínios de aplicação setorial.

de latência e 20%–40% de tokens (MICROSOFT, 2026); tais números devem ser confrontados com os ganhos de qualidade e de compressão antes da adoção.

Em contrapartida, a compressão por sub-agentes reduz o consumo do agente pai em mais de 90% (EPSILLA, 2026), superando o overhead em tarefas longas. A arquitetura de custos deve seguir *difficulty-aware routing*: modelos menores para roteamento e verificações; avançados reservados a planejamento e consolidação (GROKIPEDIA, 2026). A Tabela 4 ilustra o efeito combinado. Recomenda-se compactar quando o contexto excede 70% do limite (STACKVIV, 2026).

Tabela 4. Efeito qualitativo de decisões de arquitetura sobre custo e latência

Decisão de arquitetura	Efeito em custo	Efeito em latência	Fonte
Sub-agentes com compressão de contexto	Reduz	Neutro/Positivo	Epsilla (2026)
Orchestrator-Worker	Aumenta	Aumenta	Microsoft (2026)
Parallel Fan-Out	Aumenta	Reduz	Digital Applied (2026); Pucek (2026)
Difficulty-aware routing	Reduz	Neutro	Grokikipedia (2026)
Checkpointing/pruning a 70%	Reduz	Neutro	Stackviv (2026)

Fonte: elaborado pelo autor com base nas referências citadas.

8 Diretrizes de Engenharia para Produção

A transição de protótipos de agentes para sistemas multi-agente de produção estável exige a aplicação de rigorosas diretrizes de engenharia (WITNESSAI, 2026; STACKVIV, 2026). As subseções a seguir detalham os cinco eixos centrais.

8.1 Gerenciamento Dinâmico de Contexto

À medida que múltiplos agentes interagem, a janela de contexto satura rapidamente. Sistemas em produção devem implementar sumarização incremental e *pruning* — remoção de

metadados irrelevantes e respostas intermediárias de ferramentas. Recomenda-se compactar quando o contexto excede 70% do limite (STACKVIV, 2026), preservando artefatos de decisão e descartando o rastro de ferramentas de baixo valor.

8.2 Otimização de Custos e Latência

A arquitetura de custos deve seguir *difficulty-aware routing*: modelos menores para roteamento e verificações; avançados reservados a planejamento e consolidação (GROKIPEDIA, 2026). *Caching* de contexto e reutilização de sub-resultados reduzem o custo marginal de requisições repetitivas.

8.3 Segurança e Isolamento

Sub-agentes devem operar sob o menor privilégio, restringindo acesso a APIs e dados ao escopo da tarefa. *Circuit breakers* e cotas rígidas previnem loops infinitos (WITNESSAI, 2026). O isolamento de contexto (Claude Agent SDK, ANTHROPIC, 2026b) limita o raio de explosão de um comprometimento.

8.4 Human-in-the-Loop

Padrões suportam participação humana: observadores em *group chat*, revisores *maker-checker* e escalação em *handoff*. Pontos de verificação obrigatórios tornam a orquestração síncrona e o estado é persistido para retomada sem *replay* (MICROSOFT LEARN, 2026), recomendável em domínios de alto risco.

8.5 Observabilidade

O *stack* mínimo combina LangSmith (rastreamento), OpenTelemetry (métricas) e *dashboards* de *workflow engine*, instrumentando toda operação e todo *handoff* (SHAKUDO, 2026; APISCOUT, 2026). Sem observabilidade, a depuração de topologias dinâmicas torna-se inviável dado o não determinismo de roteamento.

9 Riscos, Limitações e Estratégias de Mitigação

Apesar dos ganhos, a adoção de *dynamic workflows* introduz riscos que devem ser explicitamente geridos. Esta seção cataloga as limitações documentadas e propõe mitigações alinhadas às diretrizes da Seção 8.

Não determinismo de roteamento. LLMs em tempo de execução podem divergir entre execuções equivalentes. *Mitigação*: abordagem híbrida com estrutura predefinida e limites explícitos (AWS, 2025); persistir traços para auditoria (SHAKUDO, 2026; APISCOUT, 2026).

Custo de tokens da orquestração. Overhead de 20%–40% em tokens e 15%–30% em latência (MICROSOFT, 2026) inviabiliza alto volume. *Mitigação*: *difficulty-aware routing* (GROKIPEDIA, 2026) e compressão (EPSILLA, 2026).

Depuração de topologias dinâmicas. Múltiplos caminhos dificultam a causa raiz. *Mitigação*: observabilidade integral (SHAKUDO, 2026; APISCOUT, 2026) e *time-travel debugging* (AGENT MARKET CAP, 2026).

Segurança e loops. Sub-agentes com privilégios excessivos e sem *circuit breaker* induzem loops ou exfiltração. *Mitigação*: menor privilégio e cotas rígidas (WITNESSAI, 2026).

Interoperabilidade. Fragmentação de protocolos (A2A, MCP) limita portabilidade. *Mitigação:* preferir frameworks com suporte nativo aos protocolos emergentemente padronizados (TZOLOV, 2026; MICROSOFT, 2026).

10 Tendências Emergentes

Seis tendências moldam o futuro (ZYLOS RESEARCH, 2026; MICROSOFT LEARN, 2026): (1) **DAGs** como linguagem de especificação (LangGraph); (2) **Orquestração *Event-Driven*** sobre barramentos (Kafka, RabbitMQ); (3) **Modelo *Actor*** (AutoGen, MAF); (4) **Roteamento Dinâmico por Dificuldade**; (5) **Orquestração Federada *cloud-edge*** via A2A; (6) **Workflows Auto-Otimizáveis** por meta-agentes. A adoção antecipada de A2A/MCP evita *lock-in* e habilita orquestração federada e auto-otimizável.

11 Roadmap de Adoção em Produção

A partir das diretrizes e riscos discutidos, propõe-se um roteiro de adoção em três fases, desenhado para minimizar risco enquanto se constrói competência organizacional.

Fase 1 — Fundação (1–4 sem.). *Stack* de observabilidade (LangSmith, OpenTelemetry) e modelo de isolamento de contexto e privilégios (WITNESSAI, 2026; SHAKUDO, 2026); iniciar com agentes generalistas simples.

Fase 2 — Orquestração Híbrida (5–10 sem.). Orchestrator-Worker ou Sequential Pipeline conforme a tarefa, com compressão por sub-agentes (EPSILLA, 2026) e *difficulty-aware routing* (GROKIPEDIA, 2026); persistir *checkpoints* (MICROSOFT LEARN, 2026).

Fase 3 — Escala e Autonomia (11+ sem.). Parallel Fan-Out/Fan-In e Dynamic Handoff para análise multi-perspectiva; *human-in-the-loop* em pontos críticos; Magentic para escopo aberto (ZYLOS RESEARCH, 2026); reavaliar topologia por custo/latência (Tabela 4).

A recomendação central é evoluir de topologias estáticas para dinâmicas de forma incremental, sustentada por observabilidade desde o primeiro dia.

12 Conclusão

Os padrões de orquestração multi-agente evoluíram de conceitos acadêmicos para pilares de arquitetura em produção empresarial. A escolha do padrão e do framework deve ser guiada por requisitos pragmáticos de latência, custos e complexidade, não por preferências de implementação. A compressão de contexto por sub-agentes consolida-se como o principal benefício prático, com reduções documentadas de mais de 90% em tokens do agente pai (EPSILLA, 2026).

Recomenda-se uma arquitetura híbrida em camadas — durabilidade (Temporal), raciocínio de agente (LangGraph), coordenação inter-serviço (Kafka + A2A) e observabilidade desde o primeiro dia — evoluindo de topologias estáticas para dinâmicas de forma incremental. Limitações atuais incluem a depuração de topologias dinâmicas, o custo de tokens da orquestração e a padronização de protocolos (A2A, MCP). Perspectivas futuras apontam para DAGs como linguagem de especificação, orquestração orientada a eventos e workflows auto-otimizáveis. O sucesso dependerá da disciplina de engenharia em observabilidade, segurança por menor privilégio e *human-in-the-loop* em pontos de alto risco.

Agradecimentos

[Os autores agradecem — A PREENCHER conforme política institucional e fontes de financiamento]

Referências

AGENT MARKET CAP. **LangGraph vs AutoGen vs CrewAI vs DSPy: The 2026 Multi-Agent Framework Decision Guide.** **Agent Market Cap**, 2026. Disponível em: <https://agent-marketcap.ai/blog/2026/04/11/langgraph-autogen-crewai-dspy-multi-agent-orchestration-2026>. Acesso em: 12 jul. 2026.

ANTHROPIC. **Claude Agent SDK: Subagents in the SDK.** São Francisco: Anthropic, 2026a. Disponível em: <https://code.claude.com/docs/en/agent-sdk/subagents>. Acesso em: 12 jul. 2026.

ANTHROPIC. **Claude Code: Orchestrate subagents at scale with dynamic workflows.** São Francisco: Anthropic, 2026b. Disponível em: <https://code.claude.com/docs/en/workflows>. Acesso em: 12 jul. 2026.

APISCOUT. **OpenAI Agents SDK: Architecture Patterns 2026.** APIScout, 2026. Disponível em: <https://apiscout.dev/guides/openai-agents-sdk-architecture-patterns-2026>. Acesso em: 12 jul. 2026.

AWS. **Agentic AI patterns and workflows on AWS.** Seattle: Amazon Web Services, 2025. Disponível em: <https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/introduction.html>. Acesso em: 12 jul. 2026.

DEVARAJAN, Vishalini. **Common Workflow Patterns for AI Agents.** GUVI Blog, 2026. Disponível em: <https://www.guvi.in/blog/common-workflow-patterns-for-ai-agents>. Acesso em: 12 jul. 2026.

DIGITAL APPLIED. **Multi-Agent Orchestration: 5 Patterns That Work in 2026.** Digital Applied, 2026. Disponível em: <https://www.digitalapplied.com/blog/multi-agent-orchestration-5-patterns-that-work>. Acesso em: 12 jul. 2026.

EPSILLA. **The 3 Essential Sub-Agent Patterns for Production-Grade AI Systems.** Epsilla Blog, 2026. Disponível em: <https://www.epsilla.com/blogs/2026-03-14-ai-sub-agent-patterns>. Acesso em: 12 jul. 2026.

GHEWARE, Rajesh. **LangGraph vs CrewAI vs AutoGen: Complete AI Agent Framework Comparison 2026.** DevOps Gheware, 2026. Disponível em: <https://devops.gheware.com/blog/po-vs-crewai-vs-autogen-comparison-2026.html>. Acesso em: 12 jul. 2026.

GROKIPEDIA. **AI Agent Orchestration.** Grokipedia, 2026. Disponível em: https://grokipedia.com/page/AI_Agent_Orchestration. Acesso em: 12 jul. 2026.

JETTHOUGHTS. **LangGraph vs CrewAI vs AutoGen: Open Source Alternatives 2025.** JetThoughts, 2026. Disponível em: <https://jetthoughts.com/blog/autogen-crewai-langgraph-ai-agent-frameworks-2025>. Acesso em: 12 jul. 2026.

LUSHBINARY. **Multi-Agent AI Orchestration Patterns: Supervisor, Swarm, Pipeline & Router.** Lushbinary, 2026. Disponível em: <https://lushbinary.com/blog/multi-agent-orchestration-patterns-supervisor-swarm-pipeline-router-guide>. Acesso em: 12 jul. 2026.

META INTELLIGENCE. **LangGraph vs CrewAI vs AutoGen: AI Agent Framework Comparison [2026].** Meta Intelligence, 2025. Disponível em: <https://www.meta-intelligence.tech/en/insight-ai-agent-frameworks>. Acesso em: 12 jul. 2026.

MICROSOFT. **AI Agent Orchestration Patterns.** Redmond: Azure Architecture

Center, 2026. Disponível em: <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/ai-agent-design-patterns>. Acesso em: 12 jul. 2026.

MICROSOFT LEARN. **Multi-agent patterns**. Microsoft Learn, 2026. Disponível em: <https://learn.microsoft.com/en-us/agents/architecture/multi-agent-patterns>. Acesso em: 12 jul. 2026.

OPENAI. **Agents SDK: Orchestration and handoffs**. São Francisco: OpenAI, 2026. Disponível em: <https://developers.openai.com/api/docs/guides/agents/orchestration>. Acesso em: 12 jul. 2026.

OPENAGENTS. **CrewAI vs LangGraph vs AutoGen vs OpenAgents: Best AI Agent Framework**. OpenAgents, 2026. Disponível em: <https://openagents.org/blog/posts/2026-02-23-open-source-ai-agent-frameworks-compared>. Acesso em: 12 jul. 2026.

PUCEK, Bartek. **Orchestration Patterns for AI Agent Workflows**. The Thinking Company, 2026. Disponível em: <https://thinking.inc/en/blue-ocean/agentic/agent-orchestration-patterns>. Acesso em: 12 jul. 2026.

ROCKB. **Microsoft Agent Framework 2026: AutoGen Successor Explained**. RockB, 2026. Disponível em: <https://baeseokjae.github.io/posts/microsoft-agent-framework-2026/>. Acesso em: 12 jul. 2026.

SHAKUDO. **Enterprise Agentic AI Workflow Patterns for 2025**. Shakudo, 2025. Disponível em: <https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/69388ca4cdb5836ee83b1ai-workflow-patterns-whitepaper.pdf>. Acesso em: 12 jul. 2026.

SHAKUDO. **Multi-Agent Systems Transform Enterprise AI in 2026**. Shakudo, 2026. Disponível em: <https://cdn.prod.website-files.com/625447c67b621ab49bb7e3e5/696fdd9dcee4a9c1agent-systems-2026-trends-whitepaper.pdf>. Acesso em: 12 jul. 2026.

STACKVIV. **Agentic AI & Multi-Agent Systems: Advanced Guide**. Stackvivi, 2026. Disponível em: <https://stackviv.ai/blog/agentic-ai-multi-agent-systems-guide>. Acesso em: 12 jul. 2026.

SUBAGENT.DEV. **Orchestration Patterns**. SubAgent.Dev, 2026. Disponível em: <https://subagent.developersdigest.tech/patterns>. Acesso em: 12 jul. 2026.

TZOLOV, Christian. **Spring AI Agentic Patterns (Part 4): Subagent Orchestration**. Spring Blog, 2026. Disponível em: <https://spring.io/blog/2026/01/27/spring-ai-agentic-patterns-4-task-subagents/>. Acesso em: 12 jul. 2026.

VELLUM AI. **The 2026 Guide to AI Agent Workflows**. Vellum AI, 2025. Disponível em: <https://www.vellum.ai/blog/agentic-workflows-emerging-architectures-and-design-patterns>. Acesso em: 12 jul. 2026.

WINBUZZER. **Anthropic Shows How to Scale Claude Code with Subagents and MCP**. WinBuzzer, 2026. Disponível em: <https://winbuzzer.com/2026/03/24/anthropic-claude-code-subagent-mcp-advanced-patterns-xcxwnb>. Acesso em: 12 jul. 2026.

WITNESSAI. **AI Agent Orchestration: Managing Multi-Agent Workflows**. WitnessAI, 2026. Disponível em: <https://witness.ai/blog/ai-agent-orchestration>. Acesso em: 12 jul. 2026.

ZYLOS RESEARCH. **Agent Workflow Orchestration Patterns: DAG, Event-Driven, and Actor Models**. Zylos Research, 2026. Disponível em: <https://zylos.ai/research/2026-04-14-agent-workflow-orchestration-patterns>. Acesso em: 12 jul. 2026.